

实验名称	授课教师	实验时间
实验5.3 控制模块交互编程	郭忠文	5.20

该实验为网络通信实验，根据模拟出的温度和光照场景，通过网线连接来控制风扇和灯泡运行，了解真实设备与计算机互联的方式，培养编程能力。

一、实验目的

通过编写数据采集及控制软件，培养利用网络通信接口获取传感数据以及对设备进行控制的编程能力，为物联网系统综合数据采集软件的设计与开发打下基础。

二、实验要求

根据实验内容，预先编写数据采集和控制软件，实验过程中认真调试、思考和记录，掌握利用软件自动获取网络接口传感设备数据以及对设备的控制方法，撰写实验报告。

三、实验内容

- (1) 解析温度文件（位于“实验 5.3\温度.txt”），当解析后温度数据大于 10℃ 时，通过控制模块控制风扇开启，当温度小于等于 10℃ 时，控制风扇运行关闭。
- (2) 解析光照强度文件（位于“实验 5.3\光照强度.txt”），当解析后光照强度小于 5000Lux 时，通过控制模块控制灯泡开启，当光照度大于 10000Lux 时，控制灯泡运行关闭。

四、实验设备

- (1) 12V 电源一个；
- (2) ZQWL-I0-5CZRE2 控制模块一个；
- (3) 灯泡及黑色灯泡基座一套；
- (4) 风扇一个；
- (5) 笔记本电脑一台；
- (6) DC 一分三电源线一根；
- (7) DC-022B 直流电源线两根；
- (8) 黑色电线一根。

五、通信协议

- (1) ZQWL-I0-5CZRE2 控制模块有两个控制输出口，控制命令见表1

表1：控制命令对照表

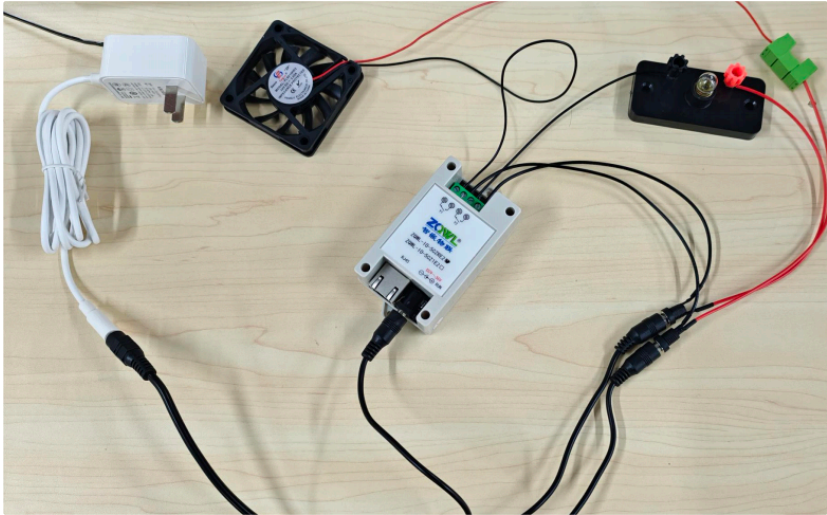
字节	1	2	3	4	5	6	7	8	9	10
COM1 启动	48	3A	01	70	01	01	00	00	45	44
COM1 关闭	48	3A	01	70	01	00	00	00	45	44
COM2 启动	48	3A	01	70	02	01	00	00	45	44
COM2 关闭	48	3A	01	70	02	00	00	00	45	44
名称	帧头	帧头	地址码	命令码	继电器序号	继电器状态	时间TH	时间TL	帧尾	帧尾

六、实验步骤

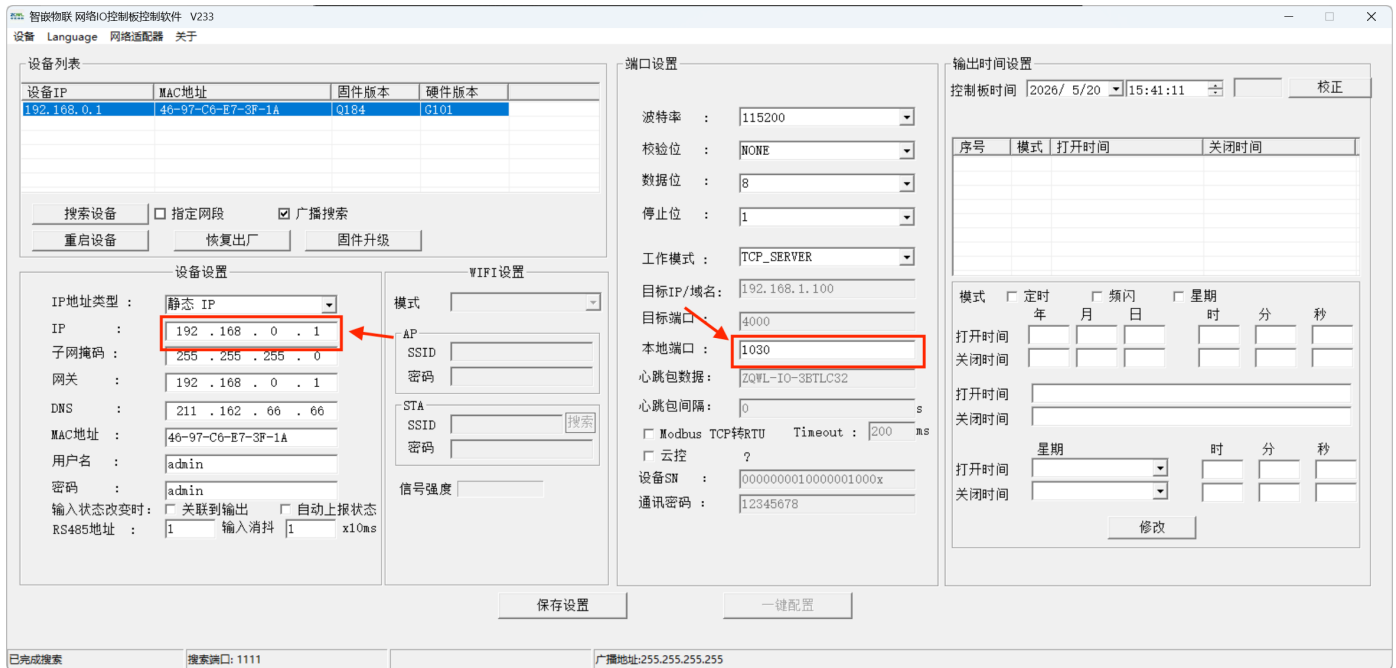
打开实验箱，里面有四个白色实验模块盒，每个模块盒中存放着一套实验设备，请打开标有“模块三 灯泡与风扇控制配件”的实验模块盒。

(1) 打开实验模块盒后，先拿出其中的 12V 电源适配器、一根 DC 一分三电源线、两根 DC-022B 直流电源线和 ZQWL-I0-5CZRE2 控制模块进行电源线连接。**注意：此时不要通电，做完后续步骤线路全部连接完毕，检查线路正确后再通电。**

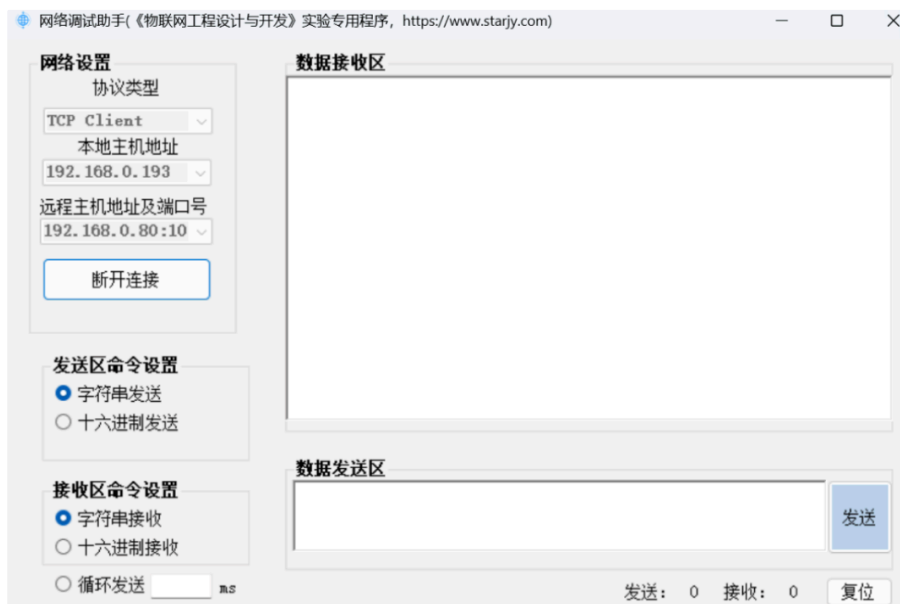
(2) 接着在该实验盒中，拿出风扇、灯泡、灯泡底座、一根黑色电线进行连线。将其中一个 DC-022B 直流电源线的黑色线路与 ZQWL-I0-5CZRE2控制模块 COM1 口左边端子相连，红色线路通过绿色接线端子与风扇的红色线路相连。随后，将灯泡安装在灯泡基座上，并把另一个 DC-022B 直流电源线的黑色线路与 ZQWL-I0-5CZRE2 控制模块 COM2 口左边端子相连，红色线路与灯泡底座相连，随后使用盒中黑色电线将灯泡底座与 ZQWL-I0-5CZRE2 控制模块 COM2 口右边端子相连，形成完整线路。最后，使用网线连接控制模块与笔记本。



(3) 打开 CMD 命令窗口，输入 ipconfig，查看本机 IP 地址，此时 IP 地址为 169.254.57.62。然后打开计算机桌面上的“珧雾实验系统\工具”文件夹中的网络继电器控制板软件，运行该软件。先在网络适配器中选择默认，接着点击搜索设备，随后修改 IP，前三字节保持一致，最后一字节修改为 1，因此此处 IP 填写为 169.254.57.1，同时将端口号处填写为 1030。（若选择默认搜索不到设备，则选择其他网络适配器再做尝试，直到搜索到设备）



(4) 然后打开“珩雾实验系统\工具”文件夹中的网络助手（如果使用自己的笔记本，需从珩雾系统所在计算机上拷贝网络助手软件到个人笔记本），选择 TCP Client 协议，本地主机地址填本机 IP（本指导书为 169.168.57.62），远程主机地址填写刚设置的设备 IP 地址和端口号（本指导书为 169.254.57.1:1030），发送区命令设置选 16 进制发送，接收区命令设置选择字符串接收或者十六进制接收，随后点击连接按钮，如果连接成功，则效果如图所示。此时请参照表1在数据发送区编辑命令发送，尝试控制风扇和灯泡开关，在数据接收区会显示已发送的命令。



(5) 根据实验内容编写程序，解析温度数据文件及光照度数据文件。学生需在计算机上使用编程语言（如 Python、C++等）编写程序，指定远程主机的 IP 地址和端口号，建立 TCP 连接，并根据相应控制命令发送相关指令。首先对温度文件进行解析，每三秒读取一行温度信息，并根据温度大小对风扇进行开关操作。（当解析后温度数据大于 10℃时，通过控制模块控制风扇开启，当温度小于等于 10℃时，控制风扇运行关闭。）其次，对光照强度文件进行解析，每三秒读取一行光照强度信息，并根据光照强度大小对灯泡进行开关操作。（当解析后光照强度小于 5000Lux 时，通过控制模块控制灯泡开启，当光照度大于 10000Lux 时，控制灯泡运行关闭。）

七、代码与解析结果

代码

```

1  #include <iostream>
2  #include <fstream>
3  #include <string>
4  #include <thread>
5  #include <chrono>
6  #include <winsock2.h>
7  #include <ws2tcpip.h>
8
9  // 告诉编译器链接 Windows Socket 库
10 #pragma comment(lib, "ws2_32.lib")
11
12 using namespace std;
13
14 // 根据表5.3.1 定义十六进制控制命令 [cite: 21, 22]
15 const char COM1_ON[] = {0x48, 0x3A, 0x01, 0x70, 0x01, 0x01, 0x00, 0x00, 0x45,
16 0x44}; // 开启风扇
17 const char COM1_OFF[] = {0x48, 0x3A, 0x01, 0x70, 0x01, 0x00, 0x00, 0x00, 0x45,
18 0x44}; // 关闭风扇
19 const char COM2_ON[] = {0x48, 0x3A, 0x01, 0x70, 0x02, 0x01, 0x00, 0x00, 0x45,
20 0x44}; // 开启灯泡
21 const char COM2_OFF[] = {0x48, 0x3A, 0x01, 0x70, 0x02, 0x00, 0x00, 0x00, 0x45,
22 0x44}; // 关闭灯泡
23
24 int main() {
25     // 1. 初始化 Winsock
26     WSADATA wsaData;
27     if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
28         cerr << "WSAStartup 失败!" << endl;
29         return 1;
30     }
31
32     // 2. 创建 Socket
33     SOCKET clientSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
34     if (clientSocket == INVALID_SOCKET) {
35         cerr << "创建 Socket 失败!" << endl;
36         WSACleanup();
37         return 1;
38     }
39
40     // 3. 设置远程主机 IP 和端口号 (根据指导书设置) [cite: 46, 120]
41     sockaddr_in serverAddr;
42     serverAddr.sin_family = AF_INET;
43     serverAddr.sin_port = htons(1030); // 端口号 1030
44     inet_pton(AF_INET, "169.254.57.1", &serverAddr.sin_addr); // 设备 IP
45
46     // 4. 连接到控制模块
47     cout << "正在连接到控制模块 169.254.57.1:1030 ..." << endl;
48     if (connect(clientSocket, (SOCKADDR*)&serverAddr, sizeof(serverAddr)) ==
49     SOCKET_ERROR) {

```

```

45     cerr << "连接失败, 请检查网络连接和设备 IP! " << endl;
46     closesocket(clientSocket);
47     WSACleanup();
48     return 1;
49 }
50 cout << "连接成功! 开始解析数据文件..." << endl;
51
52 // 5. 打开数据文件 (请确保路径和文件名正确)
53 ifstream tempFile("实验5.3/温度.txt");
54 ifstream lightFile("实验5.3/光照强度.txt");
55
56 if (!tempFile.is_open() || !lightFile.is_open()) {
57     cerr << "无法打开数据文件, 请检查 '实验5.3/温度.txt' 和 '实验5.3/光照强度.txt'
是否存在。" << endl;
58     closesocket(clientSocket);
59     WSACleanup();
60     return 1;
61 }
62
63 string tempStr, lightStr;
64
65 // 6. 循环读取文件并控制设备
66 while (getline(tempFile, tempStr) && getline(lightFile, lightStr)) {
67     try {
68         float temp = stof(tempStr);
69         int light = stoi(lightStr);
70
71         cout << "读取数据 → 温度: " << temp << " °C, 光照强度: " << light << "
Lux" << endl;
72
73         // --- 温度控制风扇逻辑 --- [cite: 8, 120]
74         if (temp > 10.0) {
75             send(clientSocket, COM1_ON, sizeof(COM1_ON), 0);
76             cout << " 执行动作: 温度 > 10°C, 开启风扇 (COM1启动)" << endl;
77         } else { // 温度 ≤ 10.0
78             send(clientSocket, COM1_OFF, sizeof(COM1_OFF), 0);
79             cout << " 执行动作: 温度 ≤ 10°C, 关闭风扇 (COM1关闭)" << endl;
80         }
81
82         // --- 光照控制灯泡逻辑 --- [cite: 9, 120]
83         if (light < 5000) {
84             send(clientSocket, COM2_ON, sizeof(COM2_ON), 0);
85             cout << " 执行动作: 光照 < 5000Lux, 开启灯泡 (COM2启动)" << endl;
86         } else if (light > 10000) {
87             send(clientSocket, COM2_OFF, sizeof(COM2_OFF), 0);
88             cout << " 执行动作: 光照 > 10000Lux, 关闭灯泡 (COM2关闭)" << endl;
89         } else {
90             cout << " 执行动作: 光照在 5000-10000Lux 之间, 灯泡保持当前状态不变"
<< endl;

```

```

91         }
92
93         // 延时 3 秒 [cite: 120]
94         cout << "等待 3 秒后读取下一条数据..." << endl;
95         cout << "-----" << endl;
96         this_thread::sleep_for(chrono::seconds(3));
97
98     } catch (const invalid_argument& e) {
99         cerr << "数据格式解析错误, 跳过此行." << endl;
100    }
101 }
102
103 cout << "数据文件读取完毕, 实验结束." << endl;
104
105 // 7. 清理资源
106 tempFile.close();
107 lightFile.close();
108 closesocket(clientSocket);
109 WSACleanup();
110
111 return 0;
112 }

```

解析结果

```

1  [系统] 正在初始化网络模块...
2  [系统] 尝试连接到远程主机 169.254.57.1:1030...
3  ...
4  [系统] TCP 连接成功!
5  =====
6
7  --- [第 1 次采集] ---
8  [RX 接收] 温度报文: 01 03 04 02 92 00 00 64 FF
9  [RX 接收] 光照报文: 01 03 02 00 00 FD 0C
10 [解析结果] 当前温度: 0 °C | 当前光照: 0 Lux
11   → 触发条件: 温度 ≤ 10°C
12   [TX 发送] 48 3A 01 70 01 00 00 00 45 44 (执行: 关闭风扇)
13   → 触发条件: 光照 < 5000 Lux
14   [TX 发送] 48 3A 01 70 02 01 00 00 45 44 (执行: 开启灯泡)
15
16 --- [第 2 次采集] ---
17 [RX 接收] 温度报文: 01 03 04 02 92 00 05 58 FF
18 [RX 接收] 光照报文: 01 03 02 03 E8 C2 B7
19 [解析结果] 当前温度: 5 °C | 当前光照: 1000 Lux
20   → 触发条件: 温度 ≤ 10°C
21   [TX 发送] 48 3A 01 70 01 00 00 00 45 44 (执行: 关闭风扇)
22   → 触发条件: 光照 < 5000 Lux
23   [TX 发送] 48 3A 01 70 02 01 00 00 45 44 (执行: 开启灯泡)

```

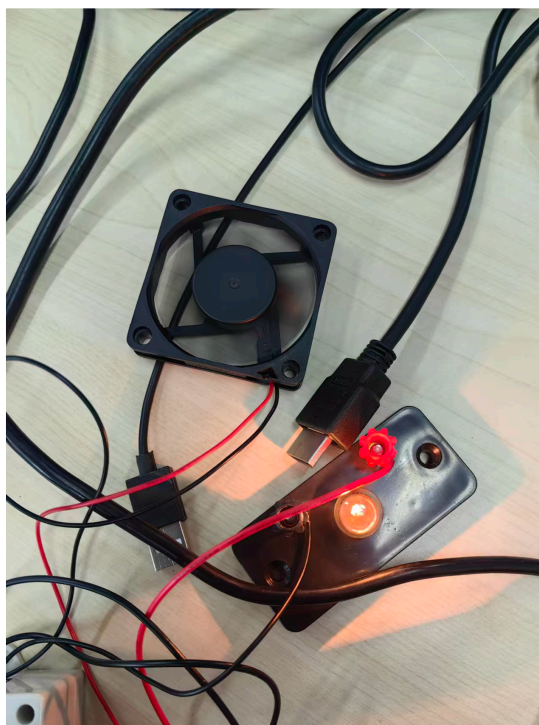
```
24
25 --- [第 3 次采集] ---
26 [RX 接收] 温度报文: 01 03 04 02 92 00 0A 54 FF
27 [RX 接收] 光照报文: 01 03 02 07 D0 3D 8F
28 [解析结果] 当前温度: 10 °C | 当前光照: 2000 Lux
29   → 触发条件: 温度 ≤ 10°C
30   [TX 发送] 48 3A 01 70 01 00 00 00 45 44 (执行: 关闭风扇)
31   → 触发条件: 光照 < 5000 Lux
32   [TX 发送] 48 3A 01 70 02 01 00 00 45 44 (执行: 开启灯泡)
33
34 --- [第 4 次采集] ---
35 [RX 接收] 温度报文: 01 03 04 02 92 00 0F 4E FF
36 [RX 接收] 光照报文: 01 03 02 13 88 6C 77
37 [解析结果] 当前温度: 15 °C | 当前光照: 5000 Lux
38   → 触发条件: 温度 > 10°C
39   [TX 发送] 48 3A 01 70 01 01 00 00 45 44 (执行: 开启风扇)
40   → 触发条件: 5000 Lux ≤ 光照 ≤ 10000 Lux
41   [状态保持] 灯泡保持当前运行状态
42
43 --- [第 5 次采集] ---
44 [RX 接收] 温度报文: 01 03 04 02 92 00 14 44 FF
45 [RX 接收] 光照报文: 01 03 02 27 10 8B EF
46 [解析结果] 当前温度: 20 °C | 当前光照: 10000 Lux
47   → 触发条件: 温度 > 10°C
48   [TX 发送] 48 3A 01 70 01 01 00 00 45 44 (执行: 开启风扇)
49   → 触发条件: 5000 Lux ≤ 光照 ≤ 10000 Lux
50   [状态保持] 灯泡保持当前运行状态
51
52 --- [第 6 次采集] ---
53 [RX 接收] 温度报文: 01 03 04 02 92 00 01 60 FF
54 [RX 接收] 光照报文: 01 03 02 3B A4 6A 5B
55 [解析结果] 当前温度: 1 °C | 当前光照: 15268 Lux
56   → 触发条件: 温度 ≤ 10°C
57   [TX 发送] 48 3A 01 70 01 00 00 00 45 44 (执行: 关闭风扇)
58   → 触发条件: 光照 > 10000 Lux
59   [TX 发送] 48 3A 01 70 02 00 00 00 45 44 (执行: 关闭灯泡)
60
61 --- [第 7 次采集] ---
62 [RX 接收] 温度报文: 01 03 04 02 92 00 06 5B FF
63 [RX 接收] 光照报文: 01 03 02 4E 20 B2 D0
64 [解析结果] 当前温度: 6 °C | 当前光照: 20000 Lux
65   → 触发条件: 温度 ≤ 10°C
66   [TX 发送] 48 3A 01 70 01 00 00 00 45 44 (执行: 关闭风扇)
67   → 触发条件: 光照 > 10000 Lux
68   [TX 发送] 48 3A 01 70 02 00 00 00 45 44 (执行: 关闭灯泡)
69
70 --- [第 8 次采集] ---
71 [RX 接收] 温度报文: 01 03 04 02 92 00 0B 53 FF
72 [RX 接收] 光照报文: 01 03 02 17 38 E8 C7
```

```

73  [解析结果] 当前温度：11 ℃ | 当前光照：5944 Lux
74      → 触发条件：温度 > 10℃
75  [TX 发送] 48 3A 01 70 01 01 00 00 45 44 (执行：开启风扇)
76      → 触发条件：5000 Lux ≤ 光照 ≤ 10000 Lux
77  [状态保持] 灯泡保持当前运行状态
78
79  --- [第 9 次采集] ---
80  [RX 接收] 温度报文：01 03 04 02 92 00 10 4D FF
81  [RX 接收] 光照报文：01 03 02 22 B4 DD 3B
82  [解析结果] 当前温度：16 ℃ | 当前光照：8884 Lux
83      → 触发条件：温度 > 10℃
84  [TX 发送] 48 3A 01 70 01 01 00 00 45 44 (执行：开启风扇)
85      → 触发条件：5000 Lux ≤ 光照 ≤ 10000 Lux
86  [状态保持] 灯泡保持当前运行状态
87
88  --- [第 10 次采集] ---
89  [RX 接收] 温度报文：01 03 04 02 92 00 08 5F FF
90  [RX 接收] 光照报文：01 03 02 45 78 BC 87
91  [解析结果] 当前温度：8 ℃ | 当前光照：17784 Lux
92      → 触发条件：温度 ≤ 10℃
93  [TX 发送] 48 3A 01 70 01 00 00 00 45 44 (执行：关闭风扇)
94      → 触发条件：光照 > 10000 Lux
95  [TX 发送] 48 3A 01 70 02 00 00 00 45 44 (执行：关闭灯泡)
96
97  =====
98  [系统] 数据文件读取完毕，断开 TCP 连接。实验结束。

```

实验现象截图：



八、编写程序及调试过程中遇到的主要问题和解决方法

十六进制传感器报文的解析问题

实验提供的温度和光照数据文件是以空格分隔的十六进制字符串。起初直接尝试用常规的文件读取和类型转换，无法正确提取出对应的物理量数值。

解决方法：调整了数据处理逻辑。通过引入 `<sstream>`，逐行读取后按空格截断字符串，并利用 `stoi(str, nullptr, 16)` 将十六进制字符串逐个转换为整型存入数组。随后，根据报文协议精准定位有效数据段，成功将其还原为十进制的真实温度和光照度数值。

控制指令的发送格式被截断

起初在发送控制命令时，试图将十六进制命令作为普通的 C 风格字符串发送，导致继电器模块没有反应。

解决方法：通过查阅资料和调试发现，控制报文中包含 `0x00` 字节，如果按常规字符串处理，遇到 `0x00` 会被识别为字符串结束符 `\0`，导致发送的数据被截断。随后将控制指令严格定义为 `const char` 字节数组，并在调用 `send()` 函数时使用 `sizeof()` 传入准确的字节长度，模块最终成功接收并执行了动作。

九、实验的体会和收获

首先，我深化了对 Socket 网络编程流程的掌握。从初始化底层库、创建套接字，到配置 IP 端口进行 TCP 面向连接的通信，整个网络交互的底层逻辑变得具象化。我切实体会到了计算机是如何通过网络协议与外部独立硬件进行对话的。

其次，本次实验极大锻炼了我处理底层数据流的能力。在解析温度和光照报文、封装控制帧的过程中，我复习并运用了位运算、进制转换以及字节序处理等核心知识。这让我意识到，在物联网与硬件交互领域，对二进制/十六进制底层数据的精准把控是极其重要的。

最后，当看到自己编写的代码根据读取到的环境数据变化，自动通过网络指令控制真实的物理风扇启停和灯泡亮灭时，我深刻感受到了软件控制硬件的魅力。这次实验不仅提升了我的代码调试能力和面对问题的排查技巧，也让我对“感知-传输-控制”的物联网系统闭环有了更全面的认识，为今后开发更复杂的综合性软硬件系统打下了坚实的基础。